



### **Unit-2 Implementing OOP concepts using C++**

- Constructors
- Static vs. non-static members
- Different types of objects
- Destructor
- Array of objects

## 1 Constructors: default, parameterized, copy, overloading

### Constructor:-

- A constructor is a special member function whose task is to **initialize the objects of its class**. It is special because its name is the same as the class name.
- The constructor is invoked whenever an object of its associated class is created.
- It is called constructor because it constructs the values of data members of the class.
- The syntax for defining a constructor is shown below:
- Syntax:

```
class   ClassName
{
    public :
        ClassName() // This is a constructor.
        {
            :
            :
        }
};
```

- Example:

```
class   test
{
    public :
        test() // This is a constructor.
        {
            cout<<"I am constructor";
        }
};

void   main()
{
    test   t; // This is call to constructor.
}
```

COURSE: SY CA & IT SEMESTER-III UNIT- 2 Object Oriented Programming Concepts

- It is possible to define a class without constructor. In such case, compiler calls a dummy constructor (i.e. which performs no action) when its object is created.

Characteristics of a constructor:

- A constructor has the following characteristics:
  - It has the same name as its class.
  - It does not have any return type.
  - It is used to initialize data members of a class.
  - It is used to allocate memory to data members of a class.
  - It is the first member function to be executed automatically when the object of the class is created.

Default Constructor:-

- “Constructors without any arguments (or parameters) are known as default constructor.”
- The syntax for defining a default constructor is shown below:

```
class  ClassName
{
    public :
        ClassName() // This is a default constructor.
        {
            :
            :
        }
};
```

There is no argument

- Example:

```
class  test
{
    public :
        test() // This is a default constructor.
        {
            cout<<"I am default constructor" ;
        }
};

void  main()
{
    test  t; // This is call to default constructor.
}
```

### Parameterized constructor:-

- “Constructor with arguments are known as parameterized constructor.”
- The syntax for defining a parameterized constructor is shown below:

```
class  ClassName
{
    public :
        ClassName(int a)  // This is a parameterized constructor.
        {
            :
            :
        }
};
```

Here, one or more parameters are allowed

- Example:

```
class  test
{
    public :
        test( int  a )  // This is a parameterized constructor.
        {
            cout<<"The value of a="<<a ;
        }
};

void  main()
{
    test  t(5) ; // This is call to parameterized constructor.
    test  s = 4 ; // Another way of calling parameterized constructor.
}
```

Output:

The value of a=5

The value of a=4

### Constructor Overloading :-

- A class have multiple constructor, and they differ only in total number of arguments or their data types or both. This is called constructor overloading.
- Example: Write a C++ program to build a class addition, which have two integer data members (x and y).

```
#include<iostream.h>
class addition
```

```
{
    public :
        int x, y, sum;
        addition() ;    // Default constructor
        addition ( int a ) ;
        addition ( int a , int b ) ;
};
class addition :: addition()
{
    x=0;
    y=0;
    sum = x + y;
    cout<<"Sum ="<<sum;
}
class addition :: addition( int a )
{
    x=a;
    y=0;
    sum = x + y;
    cout<<"Sum ="<<sum;
}
class addition :: addition(int a, int b)
{
    x=a;
    y=b;
    sum = x + y;
    cout<<"Sum ="<<sum;
}
void main()
{
    addition a1 ;    //This call default constructor, display Sum = 0
    addition a2(5) ; //This call constructor with one argument, display Sum = 5
    addition a3(5,7); //This call constructor with two argument, display Sum = 12
}
```

#### **6.4 Copy constructor:-**

- “A constructor having a reference to an object of its own class as an argument is known as copy constructor.”
- The argument of a constructor can be of any type except an object of its own class as a **value** parameter.

COURSE: SY CA & IT SEMESTER-III UNIT- 2 Object Oriented Programming Concepts

```
class test
{
    :
    test ( test t1 );
    :
}
```

This is invalid. It is an Error

But, an object of a class can be passed as a reference parameter as shown below.

```
class test
{
    :
    test ( test &t1 ); // Copy constructor.
    :
}
```

Reference to an object of the class test

- Example:

```
#include<iostream.h>
class test
{
    int i;
    public:
        test();           //default constructor
        test(test &x);    //copy constructor
        void disp();
};
test::test()
{
    i=1;
}
test::test(test &x)
{
    i=x.i+1;
}
void test:: disp()
{
    cout<<i<<endl;
}
void main()
{

    clrscr();
```

**COURSE: SY CA & IT SEMESTER-III UNIT- 2 Object Oriented Programming Concepts**

```
test t1;          //call default constructor
t1.disp();
test t2(t1);      // call copy constructor
t2.disp();
test t3(t2);      // call copy constructor again
t3.disp();
getch();
}
```

**2 Static vs. non-static members****Static Data Member:-**

A data member of a class can be qualified as static. The properties of **static** member variable are similar to that of C static variable. A static member variable has certain special characteristic. These are:

- ✓ It is initialized to zero when the first object of its class is created. No other initialization is permitted.
- ✓ Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- ✓ It is visible only within the class, but its lifetime is the entire program.

Static variables are normally used to maintain values common to the entire class. For example, a static data member can be used as a counter that records the occurrences of all the objects. Following program illustrate the use of a static data member.

Note that the type and scope of each static member variable must be defined outside the class definition .This is necessary because the static data members are stored separately rather than as a part of an object.

Example:-

```
#include<iostream.h>
class item
{
    static int count; //count is static
    int number;
    public:
    void getdata(int a)
    {
```

COURSE: SY CA & IT SEMESTER-III UNIT- 2 Object Oriented Programming Concepts

```
        number=a;
        count++;
    }
    void getcount(void)
    {
        cout<<"count:";
        cout<<count<<endl;
    }
};
int item :: count ; //count defined

int main( )
{
    item a,b,c;
    a.get_count( );
    b.get_count( );
    c.get_count( );
    a.getdata( );
    b.getdata( );
    c.getdata( );
    cout<<"after reading data : "«endl;
    a.get_count( );
    b.get_count( );
    c.get_count( );
    return(0);
}
```

The output would be

count:0

count:0

count:0

After reading data

count: 3

count:3

count:3

The static Variable count is initialized to Zero when the objects created. The count is incremented whenever the data is read into an object. Since the data is read into objects three times the variable count is incremented three times. Because there is only one copy of count shared by all the three object, all the three output statements cause the value 3 to be displayed.

COURSE: SY CA & IT SEMESTER-III UNIT- 2 Object Oriented Programming Concepts

**Static Member Function:-**

A member function that is declared static has following properties:-

1. A static function can have access to only other static members declared in the same class.
2. A static member function can be called using the class name as follows:-

class - name :: function - name;

Example:-

```
#include<iostream.h>
class test
{
    int code;
    static int count; // static member variable
public:
    void set(void)
    {
        code=++count;
    }
    void showcode(void)
    {
        cout<<"object member : "<<code<<end;
    }
    static void showcount(void)
    {
        cout<<"count="<<count<<endl;
    }
};
int test:: count;

int main()
{
    test t1,t2;
    t1.setcode( );
    t2.setcode( );
    test :: showcount ( );

    test t3;
```

COURSE: SY CA & IT SEMESTER-III UNIT- 2 Object Oriented Programming Concepts

```
t3.setcode( );  
test:: showcount( );  
t1.showcode( );  
t2.showcode( );  
t3.showcode( );  
return(0);  
}
```

**output:-** count : 2  
count: 3  
object number 1  
object number 2  
object number 3

**3 Different types of objects : constant objects, nameless objects, live objects**

**3.1 Constant Object:-**

- C++ allows to define constant object of a class.

• **Syntax:**

```
class Name const ObjectName ( Arguments );
```

Diagram labels:  
- Name of a class points to `class`  
- Name of an Object points to `ObjectName`  
- Keyword points to `const`

- Constant objects are initialized only by a constructor during object creation. Once an object is created, no member function of its class can modify its data members. They can only read the data members. Such a data members are known read only data members and object is known as **read only object** or **constant object**.

//Constant object...

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
class student
```

```
{
```

```
public :
```

```
int no;
```

```
char name[50];
student (int n, char nm[50]);
void print();
};
student :: student(int n, char nm[50])
{
    no = n;
    strcpy(name,nm);
}
void student :: print()
{
    cout<<no<<name;
}
void main()
{
    student const s1(1,"XYZ");
    clrscr();
    s1.no=5; // Error :: Can not modify constant object.
    s1.print();
    getch();
}
```

### **3.2 Nameless Object:-**

- C++ supports the creation of unnamed object. It means that in creation of an object, the name of an object need not mentioned.
- **Syntax:**

className ( Arguments ) ;

Class name      ↗      ↖      Arguments to constructor

- In above syntax, the name of object is not mentioned.
  - Passing arguments to an object is optional. If there is no argument, a default constructor is called. If there is argument, corresponding constructor is called.
  - The scope of a nameless object is limited only to the statement in which it is created because nameless objects are immediately destroyed after execution of a constructor.
  - The creation of nameless object is useful in function returning an object.
- Example:

```
#include<iostream.h>
class test
{
    public :
        int a;
        test ()
};
test :: test ()
{
    cout<<"Constructor \n";
}
void main()
{
    test () ; // This statement created as well as destroyed nameless object.
}
```

### 3.3 Live object:

- Objects created dynamically with their data members initialized during creation are known as live object. The new operator is used for creating a live object. The syntax for creating live object is as follows.

Ptr\_to\_object = **new** className ( Parameters );

- A class whose live object is to be created must have at least one constructor.

```
#include<iostream.h>
#include<conio.h>
class student
{
    private :
        int roll_no;
        char *name;
    public :
        student( int no)
        {
            roll_no = no;
            name = NULL;
        }
        student( int no, char *nm)
        {
            roll_no = no;
            name = new char [ strlen(nm)+1 ] ;
            strcpy(name,nm);
        }
        ~student()
        {
            delete name;
        }
};
void main()
{
    student *s1, *s2;
    s1 = new student( 1 ); //Live object
    s2 = new student( 1, "xyz" ); //Live object
}
```

- The syntax for destroying live object is the same as that of normal dynamic objects.  
delete object\_Name ;

The delete operator is used to release memory.

## 4 Destructor

### 4.1 Destructor:-

- “Destructor is a special member function, which is called automatically when object is destroyed. It has the same name as its class prefix by ~ (Tilde) character.”

- The syntax for defining a destructor is shown below:

```
class  ClassName
{
    public :
        ClassName( )  // This is a constructor.
        {
            :
            :
        }
        ~ClassName( )  // This is a destructor.
        {
            :
            :
        }
};
```

- A class cannot have more than one destructor. Like constructor, destructors have no return type. Unlike the constructor, the destructor does not take any arguments.
- Example:

```
class  test
{
    public :
        test( )  // This is a constructor.
        {
            cout<<"I am constructor" ;
        }
        ~test( )  // This is a destructor.
        {
            cout<<"I am destructor" ;
        }
};

void  main()
{
```

```
test t ; // This is call to constructor.
Cout<<"\n In main function";
```

```
}
```

Output:

I am constructor

In main function

I am destructor

**Differentiate between Constructor and Destructor**

|     | Constructor                                                                                             |     | Destructor                                                                                       |
|-----|---------------------------------------------------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------|
| 1.  | Constructor is a special member function, which is called automatically when object is created          | 1.  | Constructor is a special member function, which is called automatically when object is destroyed |
| 2.  | Syntax:<br><b>Note</b> : Write down syntax here.                                                        | 2.  | Syntax:<br><b>Note</b> : Write down syntax here.                                                 |
| 3.  | Example:<br><b>Note</b> : Write down example here.                                                      | 3.  | Example:<br><b>Note</b> : Write down syntax here.                                                |
| 4.  | Arguments can be passed to constructor.                                                                 | 4.  | Arguments cannot be passed to destructor.                                                        |
| 5.  | Constructors cannot be virtual.                                                                         | 5.  | Destructor can be virtual.                                                                       |
| 6.  | It has the same name as its class                                                                       | 6.  | It has the same name as its class but prefix by ~(Tilde) character.                              |
| 7.  | A class can have more than one constructor                                                              | 7.  | A class cannot have more than one destructor                                                     |
| 8.  | Constructor can be overloaded.                                                                          | 8.  | Destructor cannot be overloaded.                                                                 |
| 9.  | It is used to allocate the resources to the object of a class.                                          | 9.  | It is used to release all the resources allocated to the object of a class.                      |
| 10. | Types of constructor are:<br>- Default constructor<br>- Parameterized constructor<br>- Copy constructor | 10. | There are no any types of destructor.                                                            |

## 5 Array of Object

### 8.4 Array of object:-

- It is possible to define object variable, which is an array. Such a variable is known as array of object.
- Consider the following class.

```
class    student
{
    public:
        int    no;
        char   name[25];
        int    marks;
        void   read();
        void   print();
};

void student :: read()
{
    :
}

void student :: print()
{
    :
}
```

- The syntax for defining an array of object is shown below.

```
class    class_Name    object_Name [ size ] ;
        or
class_Name    object_Name [ size ] ;
```

- In above, class is keyword, class\_Name is the name of the class, object\_Name is any valid variable name and size is integer constant value. An array of the above class can be defined as follows.

```
class    student    s[50];  
  
        or  
  
student    s[50];
```

- The syntax for accessing its members using an index.

Class\_Name [ index ]. MemberName //for data member

Class\_Name [ index ]. MemberFunction(parameter) //for member function

- The following statement access members of the class.

```
cin>> s[4].name ; //It reads name of the 5th student.
```

```
cin>>s[2].marks ; //It reads marks of the 3rd student.
```

```
cout<< s[0].no ; // It prints number of 1st student.
```

```
S[2].read() ; //It calls read function of a class.
```

- Program: Following program reads the number, name and total marks of the n Student and displays them.

```
#include<iostream.h>  
#include<conio.h>  
class    student  
{  
    private :  
        int    no;  
        char    name[25];  
        int    marks;  
    public :  
        void read();  
        void print();  
};  
void    student :: read()
```

COURSE: SY CA & IT SEMESTER-III UNIT- 2 Object Oriented Programming Concepts

```
{
    cout<<"Student number...";
    cin<<no;
    cout<<"Student name...";
    cin>>name;
    cout<<"Student total marks...";
    cin<<marks;
}
void student :: print()
{
    cout<<"\n"<< no<<" "<<name<<" "<<marks;
}

void main()
{
    student s[50];
    int i,n ;
    clrscr();
    cout<<"Enter number of student...";
    cin>>n;
    for( i=0 ; i<n ; i++)
        s[i].read();
    cout<<"\n Student Report ";
    cout<<"\n.....";
    cout<<"\n No   Name Marks";
    cout<<"\n.....";
    for ( i=0 ; i<n ; i++)
        s[i].print();
    getch();
}
```

Input:

Enter number of student... 2  
Student number...1  
Student name...Arjun  
Student marks...50

Student number...2

Student name...Vijay  
Student marks...80

Output:

Student Report

| No. | Name  | Marks |
|-----|-------|-------|
| 1   | Arjun | 50    |
| 2   | Vijay | 80    |